

บทที่ 4

ภัย และการป้องกันภัยจากอินเทอร์เน็ต

การ exploit บางอย่างต้องการผู้ที่มีทักษะความสามารถสูงในการพัฒนา เช่น การทำให้เกิด Buffer Overflow โดยหลีกเลียงบริการบางอย่าง เช่น Secure Shell แต่การ exploit บางประเภทก็ไม่จำเป็นต้องมีทักษะสูงมากนักในการ exploit ให้สำเร็จได้ เช่นการโจมตีโดยการใส่ข้อมูลที่ผิดพลาดลงบน form ที่ปราศจากการตรวจสอบข้อมูลที่ถูกต้อง

โดยมุมมองทางวิชาการแล้วการโจมตีโปรแกรมประยุกต์สามารถแบ่งออกได้ 3 ประเภทหลักๆ ดังต่อไปนี้

1. **Syntactic Attacks** เกิดขึ้นจากการที่ระบบเชื่อในสิ่งที่โปรแกรมได้รับข้อมูลเข้ามา แล้วทำให้โปรแกรมเกิดการทำงานผิดพลาดขึ้น เช่น SQL Injection ซึ่งถือว่าเป็นตัวอย่างที่ดี เป็นต้น
2. **Semantic Attacks** เป็นการเปลี่ยนแปลงค่าข้อมูลในการส่งไปให้โปรแกรมประยุกต์ทำงาน เช่น โปรแกรมของเราต้องการให้ไฟล์ login.html เป็น argument แต่จะเกิดอะไรขึ้นถ้าเราเปลี่ยน login.html เป็น /etc/passwd
3. **Logical Attacks** คือการทำให้การทำงานของโปรแกรมไม่สามารถทำงานต่อได้ เช่น การ upload ไฟล์ที่มีขนาดใหญ่เกินไป เป็นต้น

4.1 SQL Injection

SQL Injection คือการที่ในเว็บมีการรับข้อมูลจากผู้ใช้งาน แล้วนำไปใช้ในการส่งให้ฐานข้อมูลทำงาน แล้วผู้ใช้งานที่เจอลอกโปรแกรมให้ทำงานนอกเหนือจากที่เราต้องการ หรือ หลอกให้โปรแกรมทำงานโดยการตรวจสอบเงื่อนไขบางอย่าง ซึ่งการทำ SQL Injection สามารถเกิดขึ้นได้ 3 รูปแบบคือ

- ☐ Numeric SQL Injection
- ☐ String SQL Injection
- ☐ Blind SQL Injection

4.1.1 Numeric SQL Injection

Numeric SQL Injection เป็นรูปแบบหนึ่งของการทำ SQL Injection โดยลักษณะของการ Inject นั้นจะเป็นลักษณะของการใช้ตัวเลขเท่านั้น เพื่อให้ได้มาซึ่งจุดประสงค์ที่ผู้โจมตีต้องการ

4.1.2 การป้องกัน Numeric SQL Injection

จากสถานการณ์สมมติที่ผ่านมา เราได้พบว่า การใช้ SQL Statement ที่ไม่รอบคอบอาจก่อให้เกิดการทำ SQL Injection ขึ้นได้ พิจารณาอีกครั้งกับตัวอย่างนี้

```
SELECT * FROM user_data WHERE user_id = userid
```

การใส่ค่าของ userid ลงไปนั้น อาจเป็นอะไรก็ได้ เช่น String หรือ ค่า Integer ดังนั้นถ้าเราคิดที่จะใส่ ค่าอย่างอื่นนอกเหนือจากที่ statement เพื่อทำ sql injection ก็จะสามารถเกิดขึ้นได้ทันที

```
SELECT * FROM user_data WHERE user_id = userid or 1=1
```

เพราะฉะนั้นการป้องกันที่ดีสำหรับกรณีนี้คือ

1. ใส่เครื่องหมาย ' หรือ " รอบตัวแปรที่มีการส่งค่ามาเพื่อประมวลผลบนฐานข้อมูล
2. จากข้อ 1 ควร ใส่เครื่องหมาย \ ทุกครั้งที่มีการตรวจพบเครื่องหมายที่อาจก่อให้เกิดการทำงานของ SQL Statement ที่ผิดพลาด เช่น ', " เป็นต้น
3. ตรวจสอบข้อมูลที่ผู้ใช้งานใส่ลงไปทุกครั้งว่าเป็นข้อมูลที่ถูกต้องหรือไม่ก่อนส่งไปให้ SQL Statement เพื่อให้ฐานข้อมูลประมวลผล

4.1.3 String SQL Injection

String SQL Injection เป็นรูปแบบหนึ่งของการทำ SQL Injection โดยลักษณะของการ Inject นั้นจะเป็นลักษณะของการใส่สายอักขระในการ inject เพื่อให้ได้มา ซึ่งจุดประสงค์ร้ายที่ผู้โจมตีต้องการ

4.1.4 การป้องกัน String SQL Injection

จากสถานการณ์สมมติที่ผ่านมา เราพบว่าตัวอย่างนั้นได้แก้ปัญหาไปแล้วข้อหนึ่งคือมีการใส่ " รอบตัวแปร

```
SELECT * FROM user_data WHERE first_name = 'first_name'
```

แต่ไม่มีการตรวจเครื่องหมาย ' หรือ " ก่อนดังนั้นจึงยังก่อให้เกิดปัญหา SQL Injection ได้อีกเช่นกัน

```
SELECT * FROM user_data WHERE first_name = 'first_name' or 'x'='x'
```

เพราะฉะนั้นการป้องกันที่ดีสำหรับกรณีนี้คือ

1. ควรใส่เครื่องหมาย \ ทุกครั้งที่มีการตรวจพบเครื่องหมายที่อาจก่อให้เกิดการทำงานของ SQL Statement ที่ผิดพลาด เช่น ', ' เป็นต้น
2. ตรวจสอบข้อมูลที่ผู้ใช้งานใส่ลงไปทุกครั้งว่าเป็นข้อมูลที่ถูกต้องหรือไม่ก่อนส่งไปให้ SQL Statement เพื่อทำให้ฐานข้อมูลประมวลผล

4.1.5 Blind SQL Injection

ผู้โจมตีจะลองทำ SQL Injection โดยลองใส่ข้อมูลที่ทำให้เกิดการ query SQL Statement ที่ผิดพลาดแล้วเกิดผลให้มีการแสดงข้อความที่ผิดพลาดออกมา ถ้า server แสดงข้อความที่ผิดพลาดออกมา ผู้โจมตีก็สามารถหาวิธีในการเจาะจากข้อความแสดงข้อความผิดพลาดนั้นได้ แต่ถ้า administrator มีความรอบคอบเพียงพอไม่ให้เกิดข้อความที่ผิดพลาด ก็สามารถช่วยได้เช่นกัน แต่โชคร้ายตรงที่ มันก็ยังไม่เพียงพอ ถ้า application ไม่มีการแสดงข้อมูลที่ผิดพลาด ก็อาจจะต้องเจอกับเหตุการณ์การโจมตีแบบ "Blind" SQL Injection ได้เช่นกัน

ช่องโหว่แบบ Blind SQL Injection นั้นส่วนมากจะมีจุดประสงค์ในการค้นหาข้อมูลที่เราต้องการโดยใช้เทคนิค brute force เพื่อให้ได้ข้อมูลมา ซึ่งไม่ได้เกิดขึ้นได้ง่ายนัก เราจะลองแสดงผลกระทบที่เกิดขึ้นให้เห็นว่า หากเหตุการณ์นี้เกิดขึ้น จะก่อให้เกิดความเสียหายได้อย่างไร

4.1.6 การป้องกัน Blind SQL Injection

สำหรับภาษา PHP นั้นการเกิด Blind SQL Injection จะเกิดได้ยากมากกรณีที่มีการป้องกันไว้ทั้ง 3 หัวข้อดังต่อไปนี้แล้ว

1. ใส่เครื่องหมาย ' หรือ " ครอบตัวแปรที่มีการส่งค่ามาเพื่อประมวลผลบนฐานข้อมูล
2. จากข้อ 1 ควร ใส่เครื่องหมาย \ ทุกครั้งที่มีการตรวจพบเครื่องหมายที่อาจก่อให้เกิดการทำงานของ SQL Statement ที่ผิดพลาด เช่น ', ' เป็นต้น
3. ตรวจสอบข้อมูลที่ผู้ใช้งานใส่ลงไปทุกครั้งว่าเป็นข้อมูลที่ถูกต้องหรือไม่ก่อนส่งไปให้ SQL Statement เพื่อทำให้ฐานข้อมูลประมวลผล

ถึงแม้จะเกิดยากก็จริง แต่ก็สามารถเกิดขึ้นได้ เช่นกรณีของ Mercury Board Version 1.1.1 ก็ยังสามารถทำได้แต่ต้องอยู่ภายใต้สภาวะที่ทำงาน PHP เวอร์ชันต่ำกว่า 4.0.0 และ MySQL เวอร์ชันต่ำกว่า 3.22.0 เท่านั้น โดยปัญหาอยู่ที่การใช้งาน Syntax UNION ซึ่งปัจจุบันนี้ปัญหาเหล่านั้นก็ได้หมดลงไปแล้ว

4.1.7 ตัวอย่างเว็บไซต์ที่สามารถทำ SQL Injection ได้

Ethic

“ การสาธิตการเจาะระบบและการ Injection ทั้งหมด ล้วนกระทำขึ้นเป็นกรณีศึกษา เพื่อให้เกิดความตระหนัก ถึงความสำคัญในการรักษาความปลอดภัยของระบบ Web Application และทราบถึงแนวคิดในการเจาะระบบของแฮกเกอร์ เพื่อให้เข้าใจ และสามารถนำไปใช้ปรับปรุง การป้องกันได้อย่างมีประสิทธิภาพเท่านั้น มิได้มีเจตนาเผยแพร่เพื่อให้ฝึกฝนหรือกระทำตามอัน อาจก่อให้เกิดความเสียหายแก่ ทรัพย์สินของผู้อื่นๆได้ ในสาธิตตลอดจนการศึกษาการเจาะระบบ ทั้งหมด กระทำการขึ้นเพื่อวัตถุประสงค์ทางการศึกษา และเมื่อเจาะระบบสำเร็จก็ถือว่าการศึกษ าบจบลง โดยไม่มีการแสวงหาผลประโยชน์เพิ่มเติมจากเว็บไซต์ที่สามารถเจาะได้”

รายละเอียดเกี่ยวกับเว็บไซต์

www.thaifood.org เป็นเว็บไซต์ที่ให้บริการข้อมูลทางด้านธุรกิจเกี่ยวกับอาหารใน ประเทศไทย โดยผู้ที่จะสามารถรับชมข้อมูลได้นั้นจะต้องเป็นสมาชิกของสมาคมเท่านั้น

จุดอ่อนของเว็บไซต์

จากการตรวจสอบพบช่องโหว่ในส่วนของการพิสูจน์สิทธิเพื่อล็อกอินเข้าสู่ระบบ ซึ่ง สามารถใช้คำสั่ง SQL injection ในการผ่านระบบพิสูจน์สิทธิได้ ทำให้ผู้ที่ไม่ได้เป็นสมาชิกก็ สามารถเข้าไปอ่านเอกสารทางธุรกิจได้เช่นกัน

สคริปต์ที่สามารถเจาะเข้าสู่ระบบได้

จากการทดลองเจาะระบบได้พบจุดบกพร่องที่สามารถล็อกอินเข้าสู่หน้าของสมาชิก ได้ โดยการดัดแปลงสคริปต์เพียงเล็กน้อยดังนี้

Member Login : xxx

Password: xxx' or 'x'=x

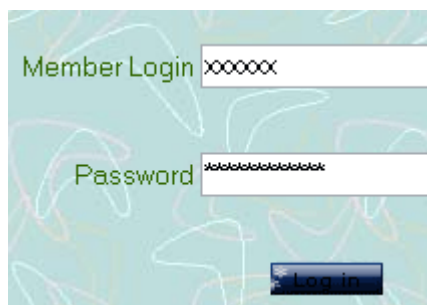
โดยให้ x สามารถใส่อะไรก็ได้

ขั้นตอนการเจาะระบบ

1. เข้าสู่เว็บไซต์ www.thaifood.org
2. login เข้าสู่ระบบโดยการทำ SQL Injection

Member Login : xxx

Password: xxx' or 'x'=x



รูปที่ 4.1 หน้า Login ของเว็บไซต์

3. หลังจาก login แล้วจะขึ้นหน้าที่แสดงรายการเอกสารต่างๆ



รูปที่ 4.2 หน้าในส่วนของผู้สมาชิกเมื่อ Login ได้สำเร็จ

4.2 Javascript injection

Javascript injection เป็นเทคนิคการแก้ไขค่าต่าง ๆ ในเพจนั้นโดยไม่ต้องเซฟ html นั้นออกมาซึ่งส่งผลให้ค่า referer ไม่มีการเปลี่ยนแปลง เพื่อใช้ในการหลบการตรวจสอบ referer จากทาง server มีวิธีการขั้นต้น 3 วิธีดังนี้:

4.2.1 Injection Basics

Javascript injections จะถูกสั่งให้จาก URL bar ที่เราใช้ใส่ address เพื่อที่จะเข้าเว็บนั้นแหละครับ โดยปกติเราจะเข้าเว็บไซต์โดยใช้ address ดังนี้ใช่ไหมครับ `http://www.google.co.th` เมื่อเราเรียก url นี้ browser จะเข้าใจว่าเราจะเรียกการทำงานผ่าน http protocol แต่รู้ไหมว่า javascript ก็สามารถรับอย่างนี้ได้เช่นกัน โดยการใส่ `javascript:xx`; ลงในช่อง url นำหน้า address ดังนี้

```
javascript:alert('Hello, World'); http://www.google.co.th
```

จะเห็นว่าเราสามารถส่ง javascript เพื่อไปรันใน google.co.th ได้ โดยคำสั่ง javascript ที่เราจะใช้ในการโจมตีมี 2 คำสั่งคือ `alert()`; and `void()`; โดยเราไม่จำเป็นต้องใส่คำสั่งทีละคำสั่ง เราสามารถใส่ หลายๆ คำสั่งได้ดังนี้ `javascript:alert('Hello'); alert('World');`

4.2.2 Cookie Editing

อันดับแรกในการแก้ค่า cookie เราต้องทำการดู cookie ทั้งหมดในเว็บนั้นก่อนว่ามีอะไรบ้าง โดยการส่ง injection ไปในไซต์ดังนี้

```
javascript:alert(document.cookie);
```

และในการ แก้ไข variable ใดๆ ในเว็บเราจะใช้คำสั่ง `void()`; ในการโจมตี ดังนี้

```
javascript:void(document.cookie="Field = myValue");
```

คำสั่งด้านบนจะทำให้ค่า cookie ที่ชื่อว่า Field มีค่าเป็น myValue การ injection cookie นี้ใช้สำหรับเว็บไซต์บางเว็บไซต์ ที่มีการใช้ cookie ในการ login หรือ การเก็บข้อมูลที่สำคัญๆ เช่น ยอดซื้อสินค้า หรือ บัตรเครดิต เป็นต้น เช่นหากการเข้าระบบถูกตรวจสอบด้วย cookie เราก็สามารถที่เข้าระบบโดยไม่ต้อง login ดังนี้

```
javascript:void(document.cookie="Authorized=yes");
```

คือให้มีการแก้ค่า cookie ที่ใช้ในการตรวจสอบให้เป็นจริง เราก็เข้าระบบได้แล้วครับ

4.2.3 Form Editing

ส่วนวิธีการสุดท้ายก็เพื่อใช้ในการหลบการตรวจสอบการแก้ค่าโดยการ check referer ที่ได้กล่าวไปในข้างต้น ทำให้เราไม่สามารถที่จะ save html ของเพจนั้นมาแก้ค่า แต่เราสามารถที่จะส่ง injection สั้นๆ เพื่อทำการแก้ค่าใน form นั้นได้ แต่ก่อนที่เราจะมาบอกวิธีการในการ hack เราจะต้องเข้าไปใน variable form ใน javascript ซะก่อนว่า form ทั้งหมดของ javascript นั้นจะเก็บเป็น array ในตัวแปร form ทั้งหมดดังนี้ forms[x] โดย X จะเป็นเลขใดๆ ที่เอาไว้แทนลำดับของ form ในเพจนั้นๆ โดย form แรกจะเริ่มตั้งแต่ 0 form แรก ก็จะถูกเรียกผ่านตัวแปร forms[0]

โดยการเรียก form แรกของเพจเราก็จะเรียกผ่านตัวแปรดังนี้ document.forms[0].to.value โดย to นั้นคือชื่อของ hidden ที่ซ่อนอยู่ โดยเราสามารถใช้อ alert ในการตรวจสอบการมีอยู่ของตัวแปรนั้นได้ ดังนี้

```
javascript:alert(document.forms[0].to.value)
```

เมื่อเราพบว่าตัวแปรที่เราต้องการจะแก้ค่านั้นมีอยู่จริงแล้วเราจะใช้คำสั่ง void ในการแก้ค่า hidden ดังนี้

```
javascript:void(document.forms[0].to.value= email@nhacks.com_CLOAKING)
```

โดยคำสั่งด้านบนจะเปลี่ยนค่าตัวแปร hidden ที่ชื่อ email ที่อยู่ใน form เป็น "email@nhacks.com_CLOAKING" โดยเราสามารถใช้อคำสั่ง alert(); ตามเพื่อตรวจสอบการเปลี่ยนแปลงค่า hidden

การตรวจสอบค่าฝั่ง client โดยใช้ javascript ไม่มีความปลอดภัยดังนั้นการตรวจสอบค่าใดๆ ในเว็บ ควรทำที่ฝั่ง server ด้วยเพื่อตัดปัญหาเรื่องนี้ แต่ปัญหานี้ไม่พบใน firefox เกิดกับเฉพาะ IE

4.2.4 การป้องกันการเกิด Javascript Injection

1. เราพบว่าสาเหตุหนึ่งที่เกิด Javascript Injection ก็เพราะโปรแกรมที่ programmer พัฒนาขึ้นมีการส่งค่าตัวแปรที่สำคัญๆ ผ่าน hidden field ดังนั้นเราจึงไม่ควรมีการเก็บค่าที่สำคัญๆ ไว้ทางฝั่ง Client ควรจะเก็บไว้ประมวลผลที่ฝั่งของ Server เท่านั้น
2. การปิดหรือยกเลิกการทำงานของ Javascript ของ Browser ก็สามารถป้องกันได้ แต่อาจทำให้ Javascript ที่สำคัญตัวอื่นไม่สามารถทำงานตามไปด้วย

4.3 Cross-Site Scripting (XSS)

Cross-Site Scripting จะเกิดขึ้นเมื่อเวลาที่เรป้อนค่าเข้าไปใน Web Browser เพื่อส่งให้กับ Web Server เช่น การป้อน Username และ Password หรือ การก็ย่ข้อมูลลงใน Web Board หลังจากที่เรากดปุ่ม Submit หรือกดปุ่ม Enter เพื่อส่งข้อมูลให้ Web Server ทาง Web Server จะสร้าง Web Page ตอบกลับมายัง Web Browser ของเราในลักษณะที่เป็น Dynamic Content เช่น ถ้าเรป้อนชื่อผิด Web Server ก็จะแจ้งว่าชื่อนั้นไม่มีในระบบ เป็นต้น

หาก Web Server มีการนำข้อมูลที่เรป้อนใส่ลงไปใน Web Page กลับมาแสดงเป็นผลลัพธ์ให้เราเห็นหลังจากที่เราได้ป้อนข้อมูลลงไป แล้ว Hacker สามารถป้อนข้อมูลที่ไม่ใช่ข้อมูลปกติลงไปในช่อง Input เช่น ป้อนข้อมูลเข้าไปในรูปแบบของ JavaScript ซึ่งสามารถทำงานตามที่ Hacker ต้องการได้ เช่น สามารถขโมย Cookie ที่อยู่ในเครื่อง PC ของเราส่งกลับไปหา Hacker ได้

ช่องโหว่ในลักษณะ Cross-Site Scripting นี้ เราจะเห็นได้บ่อยๆ ใน Search Engine ที่มีการทวน Search Keyword ที่เรป้อนลงไป หรือใน Web Site ที่มีการทวน String ของข้อมูลที่เรป้อนลงไป ในลักษณะของ error message หรือ รูปแบบของ Web Form ที่มีการทวนข้อมูลหลังจากที่เราได้ก็ย่เข้าไปในตอนแรก และ พวก Web Board ที่อนุญาตให้ User สามารถเข้ามา POST ข้อมูลได้ เป็นต้น

เมื่อ Hacker พบว่า Web Site มีช่องโหว่ให้สามารถทำ Cross-Site Scripting ได้ Hacker ก็จะเขียน Script ที่สามารถดูข้อมูลส่วนตัว ของเราที่เก็บไว้ในเครื่องเราเองในลักษณะที่เป็น Cookie ส่งกลับไปหา Hacker ให้ Hacker สามารถดูข้อมูลของเราได้อย่างง่ายดาย หรือ ส่งพวก Malicious Script แปลกๆ มา Run บนยังเครื่องเราตามที่ Hacker ต้องการก็สามารถทำได้หาก Web Browser ของเรานั้น อนุญาตให้ Run Script ต่างๆ ได้ เช่น JAVA Script เป็นต้น

4.3.1 วิธีการป้องกันช่องโหว่ Cross-Site Scripting (XSS)

วิธีสำหรับ User ที่ใช้งานทั่วไป

- ถ้าคุณสังเกตเห็นช่องโหว่ cross-site scripting ในเว็บไซต์ใดโปรดแจ้งให้เว็บมาสเตอร์ของเว็บไซต์นั้นได้รับทราบและส่งสำเนา (cc) ให้ CERT Coordination Center ด้วย
- วิธีการที่ดีที่สุดในการป้องกันสำหรับ User คือการปิดการทำงานของ scripting ของเบราว์เซอร์ถ้าไม่จำเป็นต้องใช้ อย่างไรก็ตามวิธีนี้ก็ไม่สามารถป้องกัน HTML ที่มัลแวร์ที่ซ่อนไว้ได้ คุณจึงควรป้องกันโดยการเข้าใช้เว็บเพจที่เกี่ยวข้องนั้นโดยตรง แทนที่จะคลิกลิงค์จากแหล่งที่คุณไม่รู้จักรหรือไม่น่าเชื่อถือ เช่น การไม่ไว้วางใจลิงค์ที่อยู่ในข้อความอีเมลที่พาไปยังเว็บไซต์ธนาคารของคุณ ถ้าคุณจำเป็นต้องเข้าไปยังเว็บไซต์ของธนาคารจริง ๆ ก็ควรไปยังเว็บไซต์นั้นโดยตรง และต้องระมัดระวังให้ดีเมื่อจะต้องป้อนข้อมูลส่วนตัวด้วย

วิธีสำหรับ Programmer

- เว็บมาสเตอร์ก็สามารถช่วยได้เช่นกัน โดยตรวจสอบว่า ไม่มีหน้าเว็บที่ส่งกลับสิ่งที่ผู้ใช้ป้อนเข้าไป โดยไม่มีการตรวจสอบความถูกต้อง นอกจากนี้แล้วยังควรสนับสนุนให้ผู้ใช้ ปิดการทำงานของสคริปต์ (scripting)
- สำหรับวิธีที่ดีที่สุดในตอนนี้เป็นคือ

1. ไม่อนุญาตโค้ด HTML ทุกชนิดโดยทำ Escape ตัวแปรที่ input เข้าไปทุกครั้ง

ตารางที่ 4.1 ตัวอย่างโค้ดการทำ Escape โดย Javascript

```
<script language='JavaScript' type='text/javascript'>
<!--
function convertToHexHTML(num) {
    var hexhtml = "";
    for (i=0;i<num.length;i++)
        hexhtml += "&#x" + num.charCodeAt(i).toString(16).toUpperCase() + ";";
    return hexhtml;
}
//-->
</script>
```

ตารางที่ 4.2 ตัวอย่างโค้ดการทำ Escape โดย PHP

```
<?php
function hexentities($str) {
    $return = "";
    for($i = 0; $i < strlen($str); $i++) {
        $return .= '&#x'.bin2hex(substr($str, $i, 1)).';';
    }
    return $return;
}
?>
```

2. อนุญาตเฉพาะ Tag ที่กำหนดเท่านั้นในการแสดงผล

ตารางที่ 4.3 ตัวอย่างโค้ดการอนุญาตเฉพาะ Tag ที่กำหนดในการแสดงผล และกรอก Script ออกจาก HTML Code

```
<?php

### Allow Tags ###
$allowed_tags = "<textarea>,<a>,<h1>,<h2>,<h3>,<h4>,<h5>,<h6>,<br>,<ol>,<i>,<b>,<table>,<tr>,<td>,<form>,<p>,<center>,<small>,<input>,<select>,<option>,<ul>,<li>,<blockquote>,<strong>";

### Test Tags ###
$text = '<i>Test paragraph.</i> <strong>Other text</strong>
<SCRIPT SRC=http://ha.ckers.org/xss.js></SCRIPT>';

### Not Allow All Tags ### กรอง Script และ Tag ออกทั้งหมด
echo "Not Allow All Tags = ";
echo strip_tags($text);
echo "<br>Allow Some Tags = ";

### Allow Some Tags ###อนุญาตเฉพาะบาง Tag เท่านั้นที่จะสามารถใช้งานได้
```

```
echo strip_tags($text,$allowed_tags);
```

```
?>
```

4.3.2 ตัวอย่างเว็บไซต์ที่สามารถทำ XSS ได้

Ethic

“ การสาธิตการเจาะระบบและการ Injection ทั้งหมด ล้วนกระทำขึ้นเป็นกรณีศึกษา เพื่อให้เกิดความตระหนัก ถึงความสำคัญในการรักษาความปลอดภัยของระบบ Web Application และทราบถึงแนวคิดในการเจาะระบบของแฮกเกอร์ เพื่อให้เข้าใจ และสามารถนำไปใช้ปรับปรุง การป้องกันได้อย่างมีประสิทธิภาพเท่านั้น มิได้มีเจตนาเผยแพร่เพื่อให้ฝึกฝนหรือกระทำตามอัน อาจก่อให้เกิดความเสียหายแก่ ทรัพย์สินของผู้อื่นๆได้ ในสาธิตตลอดจนการศึกษาการเจาะระบบ ทั้งหมด กระทำการขึ้นเพื่อวัตถุประสงค์ทางการศึกษา และเมื่อเจาะระบบสำเร็จก็ถือว่าการศึกษา จบลง โดยไม่มีการแสวงหาผลประโยชน์เพิ่มเติมจากเว็บไซต์ที่สามารถเจาะได้”

รายละเอียดเกี่ยวกับเว็บไซต์

www.bjctogether.com เป็นเว็บไซต์ที่ให้บริการข้อมูลข่าวสารเกี่ยวกับนักแสดงชื่อดังชาว เกาหลี โดยในเว็บไซต์นั้นมี Webboard สำหรับแสดงความคิดเห็น โดยผู้ที่สามารถ post ได้นั้น จะต้องเป็นสมาชิกเท่านั้น

จุดอ่อนของเว็บไซต์

จากการตรวจสอบพบช่องโหว่ในส่วนของการพิสูจน์สิทธิ์เพื่อล็อกอินเข้าสู่เว็บบอร์ด ซึ่ง สามารถใช้คำสั่ง SQL injection ในการผ่านระบบพิสูจน์สิทธิ์ได้ นอกจากนั้นเว็บบอร์ดยังสามารถ ทำ Cross-Site Scripting ได้ด้วยซึ่งทำให้สามารถเก็บข้อมูลของผู้ที่เข้ามาชมเว็บบอร์ดได้ด้วย

ขั้นตอนการเจาะระบบ

1. เข้าสู่เว็บไซต์ www.bjctogether.com
2. เข้าสู่หน้า Webboard เราจะสังเกตเห็นว่ามีชื่อของ admin ที่หน้าสนใจคนนึงชื่อว่า tigger



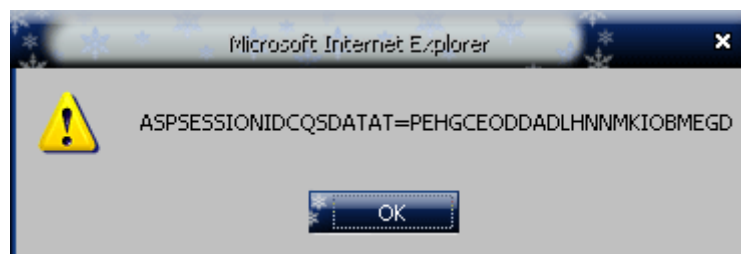
Webboard

sign in | new member

no	subject	name	date	hit	comment
★	ของที่ระลึกที่สำนักงานการท่องเที่ยว BYtogether-Special Gift from Korean Air to	BYtogether	16/12/2005 11:12:17	745	7
★	2005 Road of Sharing	BYtogether	29/11/2005 16:43:13	1704	8
★	ขอเชิญรับชม... ที่ กรุงเทพมหานคร ก่อนโผล่ชื่อทอม	BYtogether	21/9/2004	1418	0
★	คำตอบสำหรับบางคำถามที่ถามมา...	tigger	22/11/2005 20:14:38	913	4
★	ข้อมูล April Snow - Production Note จาก Major CinePictures	tigger	21/9/2005 15:06:42	3557	21
1547	MV Hotelier คุณ Frank ล้วนสุขะ	onlybyj	2/3/2005	664	1
1546	[OLD VOD]Scandal	jumbo	20/12/2005 22:05:15	43	0

รูปที่ 4.3 หน้า Webboard ของเว็บไซต์

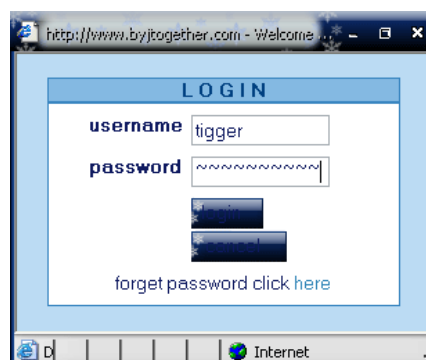
3. เรียกดู cookie โดยใช้ `javascript:alert(document.cookie);`



รูปที่ 4.4 หน้าต่างป๊อปอัพแสดง Cookie เมื่อทำ Javascript Injection

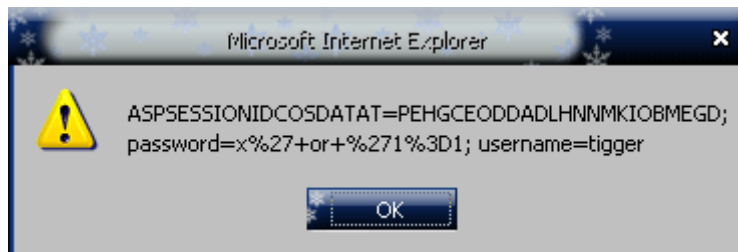
4. Login โดยทำ SQL Injection

ให้ username เป็น **tigger** และ password เป็น **x' or '1=1**



รูปที่ 4.5 หน้าต่าง login ของเว็บไซต์

5. จากนั้นลองดู cookie อีกครั้ง พบว่ามีข้อมูลเพิ่มขึ้นเป็น username และ password โดยค่าของทั้งสองตัวนั้นไม่ได้มีการเข้ารหัสอะไรไว้เลย

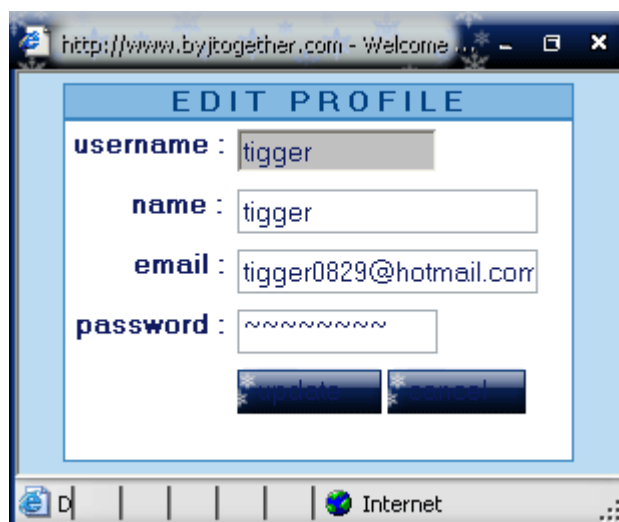


รูปที่ 4.6 หน้าต่างป๊อปอัพแสดง Cookie เมื่อทำ Javascript Injection ซึ่งแสดงค่า Session, Username, Password

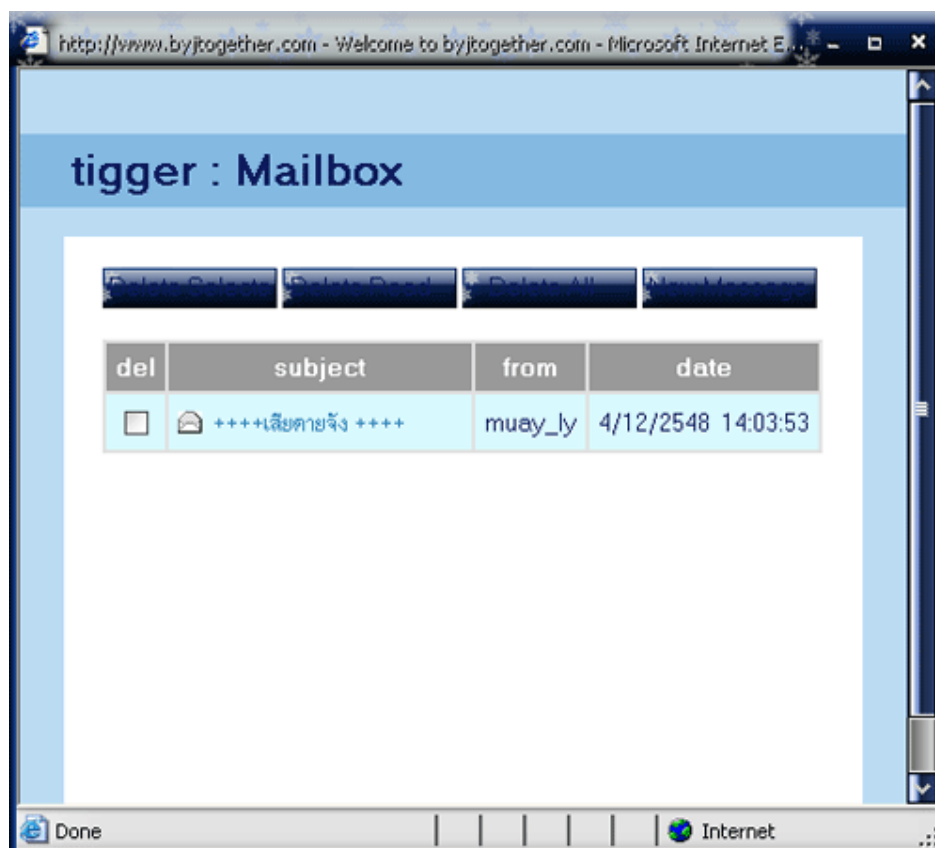
6. ลองดูที่แถบเมนูพบว่าเราสามารถมีสิทธิแก้ไขข้อมูลส่วนตัว ตั้งกระทู้ใหม่ รวมถึงสามารถอ่าน mailbox ได้ด้วย

[sign out](#) | [edit profile](#) | [mailbox\(0\)](#) | [new topic](#)

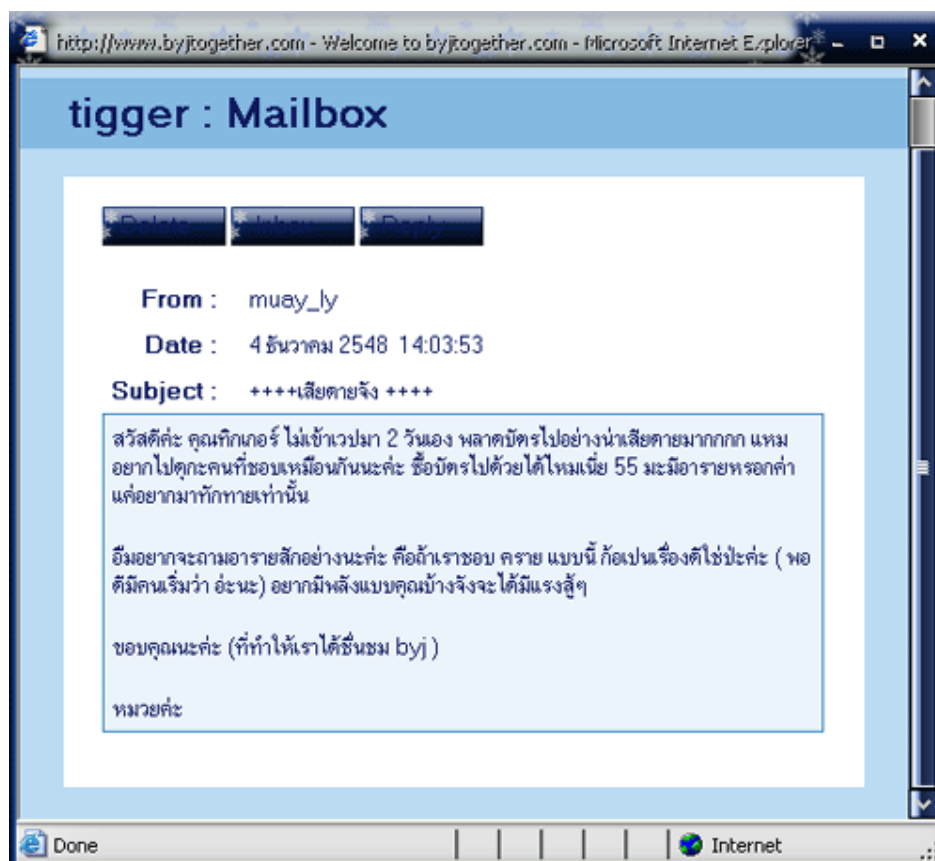
รูปที่ 4.7 เมนูของผู้ที่มีสิทธิเป็นสมาชิก



รูปที่ 4.8 หน้าแก้ไข Profile



รูปที่ 4.9 หน้า Mailbox ของสมาชิก



รูปที่ 4.10 หน้าแสดงข้อความใน Mailbox

7. ถ้าต้องการโพสข้อมูลลง webboard ชื่อของผู้ที่โพสก็จะเป็น username ที่เรา login เข้าไปนั่นเอง จากนั้นได้ลองโพส script `<SCRIPT>alert(document.cookie);</SCRIPT>` ลงไปผ่านเว็บบอร์ด

subject: `<SCRIPT>alert(document.cookie);</SCRIPT>`

detail:

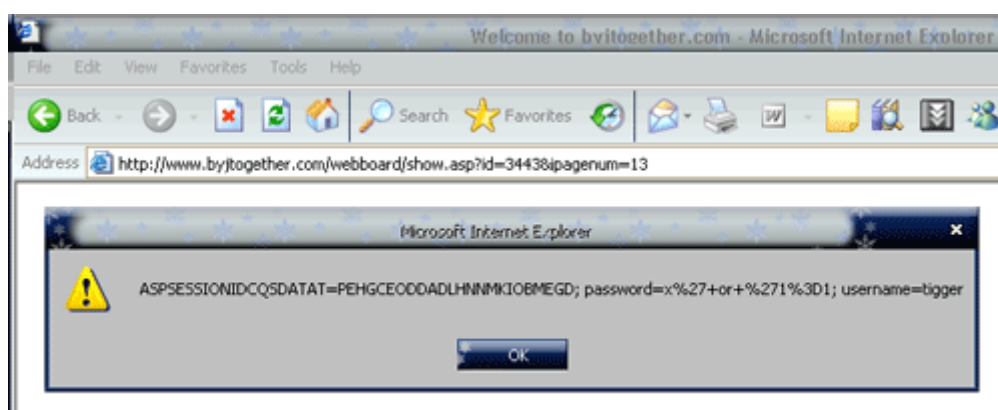
sound:

vod:

name: tigger

รูปที่ 4.11 ฟอรัมโพสข้อมูลลง Webboard

8. พบว่าเมื่อมีการ reload หน้านั้นก็ยังสามารถทำงาน script นั้นได้ แสดงว่าเว็บบอร์ดนี้สามารถแทรก script ลงไปได้



รูปที่ 4.12 หน้าต่างป๊อปอัพแสดง Cookie เมื่อทำ Javascript Injection

9. ดังนั้นจึงได้แทรก script เพื่อดักข้อมูลของ member ออกมา ซึ่งข้อมูลใน cookie นั้นเราสามารถรู้ทั้ง username และ password ของผู้ที่ login มาได้ด้วยเช่นกัน

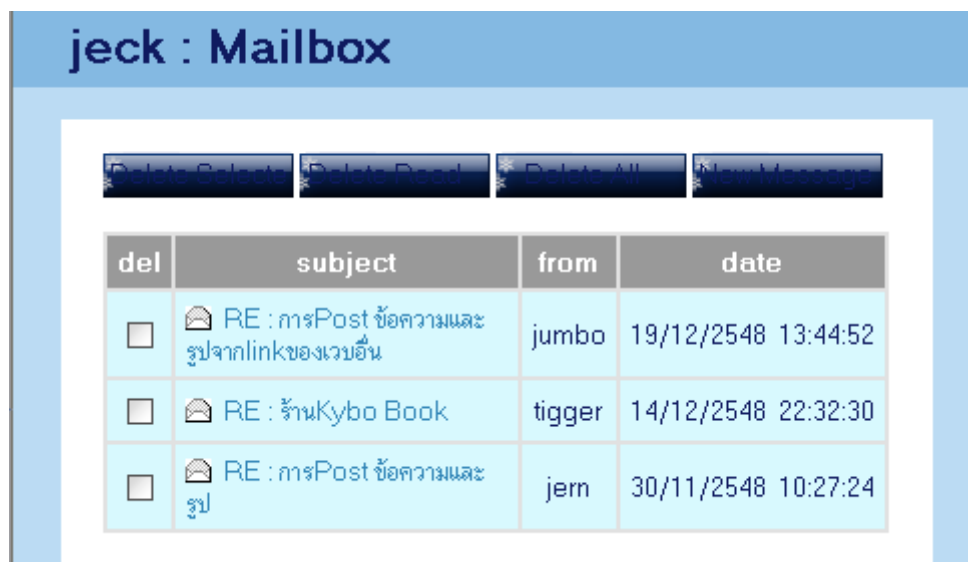
Show @ [REDACTED]

date	21-12-2005
time	2:25:0
ip	not available
codename	Mozilla
version	4.0
platform	Win32
pagesviewed	12
COOKIE	password=jeck; username=jeck; ASPSESSIONIDASRACSBT=IDPJGMBAGEBKNKCIOFCHBCKI

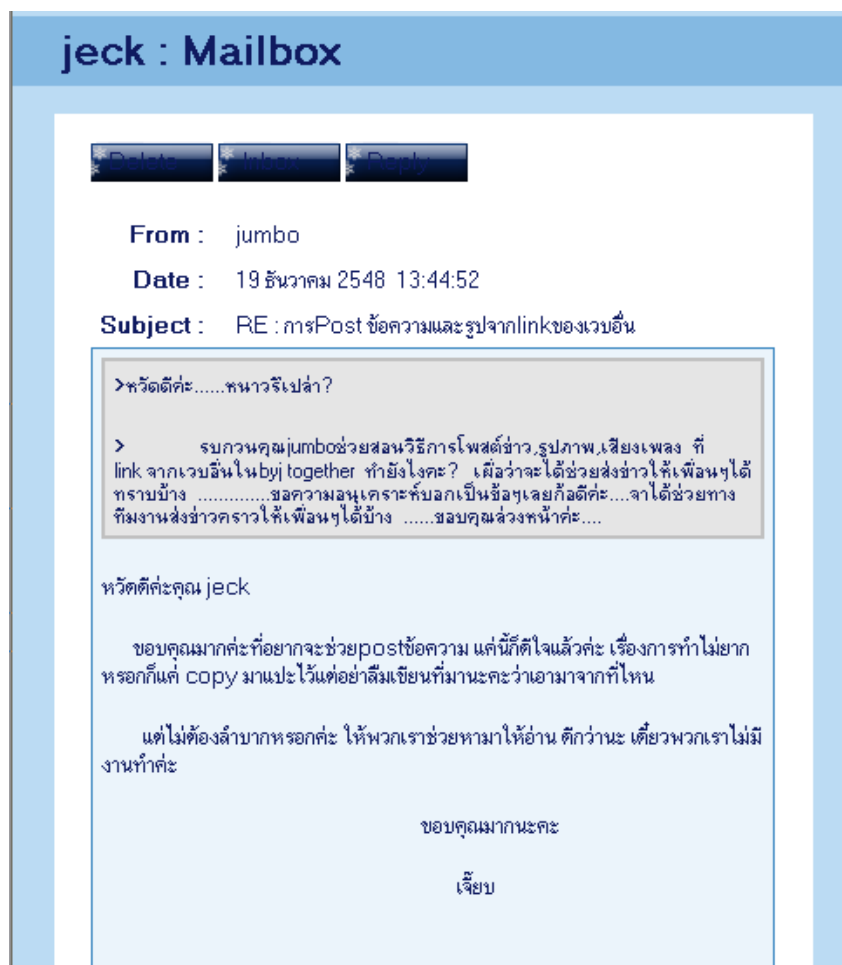
date	21-12-2005
time	2:23:34
ip	not available
codename	Mozilla
version	4.0
platform	Win32
pagesviewed	12
COOKIE	password=jeck; username=jeck; ASPSESSIONIDASRACSBT=IDPJGMBAGEBKNKCIOFCHBCKI

รูปที่ 4.13 หน้าแสดงข้อมูลที่ได้จากการทำ Cross-Site Scripting

10. จากนั้นลองนำ username และ password ที่ได้จากการทำ XSS มา login ก็จะสามารถ login ได้ถูกต้องเช่นกัน



รูปที่ 4.14 หน้า Mailbox ของสมาชิก



รูปที่ 4.15 หน้าแสดงข้อความใน Mailbox

4.4 การเข้าแย่งเพื่อครอบครองเซสชัน (Session Hijacking)

การเข้าแย่งเพื่อครอบครองเซสชันไม่ใช่เรื่องใหม่สำหรับวงการความปลอดภัยคอมพิวเตอร์ คำๆนี้มักถูกใช้บ่อยครั้งเพื่ออธิบายถึง กระบวนการในการเข้าครอบครองคอนเนกชัน การสื่อสารของโปรโตคอล TCP จากผู้อื่นด้วยภาคเคเลลำดับขั้นตอนการรับส่งข้อมูล ในกระบวนการนี้ แฮกเกอร์จะเข้าครอบครองคอนเนกชัน TCP ที่ได้สร้างแล้ว ในแง่ความปลอดภัยบนเว็บแอปพลิเคชัน การเข้าแย่งเพื่อปลอมแปลงเซสชัน จะหมายถึงการเข้าครอบครองเซสชันของเว็บแอปพลิเคชันอย่างไม่ต้อง

แอปพลิเคชันที่รองรับผู้ใช้หลายคนพร้อมกันได้มีการใช้คอนเซ็ปต์ของ “เซสชันของผู้ใช้” ด้วยภายในโดยผู้ใช้แต่ละคนจะโต้ตอบกับแอปพลิเคชัน ผ่านทางแต่ละเซสชันของตนเองต่างหาก แอปพลิเคชันจะติดตามสถานะของผู้ใช้แต่ละคนที่กำลังใช้งานแอปพลิเคชัน โดยผ่านทางเซสชัน ซึ่งเซสชัน (session) มีประโยชน์มากสำหรับการแบ่งแยกพฤติกรรมของผู้ใช้แต่ละคน

4.4.1 การจัดการกับ Session

การจัดการกับ Session นั้นไม่ได้หมายถึงการจัดการสถานะของ session เท่านั้น แต่รวมไปถึงการจัดการข้อมูลของ Client ในแต่ละ User ด้วย ซึ่งข้อมูลเหล่านี้เรียกว่า Session Data หากเราใช้ฟังก์ชันการจัดการ Session โดยทั่วไปของ PHP แล้ว Session Data จะถูกเก็บไปในตัวแปร \$_SESSION

ตารางที่ 4.4 แสดงตัวอย่างการใช้งาน Session

```
<?php
session_start();
$_SESSION['foo'] = 'bar';
?>
<a href="session_continue.php">session_continue.php</a>
```

สมมติว่าคลิกแล้วลิงค์ไปที่ session_continue.php จะทำให้ script ที่รับหน้าต่อไปสามารถ ใช้ session นี้ได้เช่นกัน

ตารางที่ 4.5 แสดงตัวอย่างการใช้งาน Session

```
<?php
session_start();
echo $_SESSION['foo']; /* bar */
?>
```

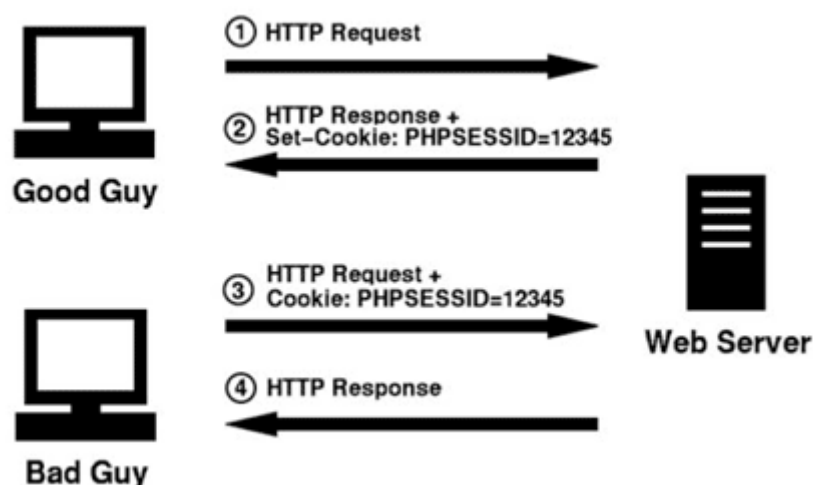
ปัญหาด้านความปลอดภัยที่สำคัญในการเกิด Session Hijacking อยู่ที่การแสดงผลของตัวแปร ออกมาดังตัวอย่างที่ 2 ซึ่งเกิดจากความไม่เข้าใจว่า PHP นั้นได้ทำอะไรบ้างสำหรับ Session สุดท้ายก็จะไม่สามารถหาเหตุผลได้ว่า Program ที่เราเขียนนั้นมีข้อผิดพลาดอยู่ที่ใด

4.4.2 การเกิด Session Hijacking

จากปัญหาที่เกิดขึ้นซึ่งเราเข้าใจว่า PHP ได้จัดการกับ session ไว้อย่างปลอดภัยแล้ว ในทางตรงข้าม ซึ่งข้อเท็จจริงแล้วผู้พัฒนาโปรแกรมต้องเป็นคนกำหนดระดับความปลอดภัยเอง จึงจะถูกต้อง โดยจากตัวอย่างที่ผ่านมาซึ่งเป็นวิธีที่ไม่ดีนัก แต่ก็ไม่ได้มีวิธีที่ดีที่เสถียรสำหรับทุกคน เช่นกัน เพื่อทำความเข้าใจการทำงานของ Session ลองพิจารณาเหตุการณ์ดังนี้

- Good Guy เข้ามาใช้บริการที่ `http://www.example.org/` และ logs in ด้วย
- `example.org` เซ็ต cookie, `PHPSESSID=12345`
- Bad Guy เข้ามาใช้บริการที่ `http://www.example.org/` และกำหนดค่า cookie เป็น, `PHPSESSID=12345`
- `example.org` เข้าใจผิดว่า Bad Guy เป็น Good Guy แล้วส่งข้อมูลต่างๆ ให้ Bad Guy

เหตุการณ์นี้แสดงเป็นรูปภาพได้ดังนี้



รูปที่ 4.16 รูปแสดงการเกิด Session Hijacking

4.4.3 ขั้นตอนการเข้าแย่งเพื่อครอบครองเซสชัน (Session Hijacking)

1. หาสถานะของตัวเลขระบุเซสชัน (Session ID)

พยายามหาสถานะของตัวเลขระบุเซสชันที่วิ่งไปมาระหว่างการเชื่อมต่อ ทดสอบความถี่การจดจำสถานะของ cookies, session cookies, URL Parameter และ hidden fields

2. ถอดรหัสข้อมูลของสถานะที่ได้มา

จากข้อแรกเราสามารถหาสถานะของตัวเลขระบุเซสชัน และวิเคราะห์สิ่งที่ได้มาได้แล้ว ถ้ายังไม่เข้าใจในสิ่งที่ได้มา เราอาจดูเป็นแผนภาพเพื่อให้เข้าใจมากขึ้น ลองตัดสินใจและทดสอบค่าของตัวเลขระบุเซสชันว่าจะเป็นค่าใดต่อไปนี้

- Incremental Value
- Date and Timestamp
- Static Value
- Pseudorandom Value
- Profile Information

- Server IP Address
- Client IP Address
- Two-Byte Numbers

3. ทดสอบค่าของตัวเลขระบุเซสชันต่างๆที่ได้มา

ลองใช้ค่าที่เราถอดออกมาได้เพื่อทดสอบเหตุการณ์ที่จะเกิดขึ้น เมื่อมีการเปลี่ยนตัวเลขระบุเซสชัน โดยจะนำตัวเลขระบุเซสชันจากการ login เพื่อใช้งานของผู้ใช้งานหลายๆคนมาแทนที่เดิม แล้วดูว่าผลกระทบที่เกิดขึ้นเมื่อมีการแก้ตัวเลขระบุเซสชันแล้วจะเป็นอย่างไร

4. ดัดแปลงสถานะของตัวเลขระบุเซสชัน

ถ้าเราสามารถถอดรูปแบบตัวเลขระบุเซสชันได้ เราก็สามารถดัดแปลงตัวเลขระบุเซสชัน เพื่อปลอมแปลงหรือ ขโมยเซสชันของผู้อื่น โดยอาจจะเปลี่ยนข้อมูล หรือ เพิ่มขีดความสามารถของตนเองในการใช้งานแอปพลิเคชันนั้นได้ด้วย

4.4.4 การป้องกันการเข้าแย่งเพื่อครอบครองเซสชัน (Session Hijacking)

สำหรับการป้องกันไม่ให้เกิด Session Hijacking นั้นมีเทคนิคอยู่หลายเทคนิคแต่ละคนขึ้นอยู่กับโปรแกรมที่ออกแบบ และความระมัดระวังแล้วแต่ตัดสินใจ

HTTP/1.1 request ที่ง่ายที่สุดที่สามารถส่งไปได้คือ

```
GET / HTTP/1.1
Host: www.example.org
```

ถ้า Client ส่ง SessionID เช่น PHPSESSID มันอาจถูกส่งผ่าน Header ไปได้เช่น

```
GET / HTTP/1.1
Host: www.example.org
Cookie: PHPSESSID=12345
```

หรืออีกทางหนึ่งคือการส่งผ่าน URL เช่น

```
GET /?PHPSESSID=12345 HTTP/1.1
```

```
Host: www.example.org
```

เราอาจจะใช้ method POST ในการใช้งานก็ได้ด้วยเช่นกัน แต่อาจทำให้เห็นภาพไม่ชัดเจนนักในการส่ง-รับข้อมูล

เราพบว่าผู้โจมตีสามารถโจมตีเราโดยการปลอมแปลง SessionID ได้ดังนั้น เราจึงต้องหาวิธีที่ดีกว่านั้น อาจจะป้องกันโดยซ่อน SessionID หรือ ทำให้ค่า SessionID สามารถเดาได้ยากก็ได้ แต่โดยปกติแล้วค่า SessionID ที่ PHP ออกมาให้นั้นก็มาจากการสุ่มอยู่แล้ว ดังนั้นเหตุการณ์ที่เกิดขึ้นจึงไม่อาจควบคุมได้ง่ายนัก

วิธีหนึ่งในการป้องกันการเกิด Session Hijacking คือการตรวจสอบค่า Option ต่างๆของ Header ที่มี เช่น User-Agent, Accept, Accept-Charset, Accept-Language หรือ Reffer เป็นต้น

```
GET / HTTP/1.1
```

```
Host: www.example.org
```

```
Cookie: PHPSESSID=12345
```

```
User-Agent: Mozilla/5.0 Galeon/1.2.6 (X11; Linux i686; U;) Gecko/20020916
```

```
Accept: text/html;q=0.9, */*;q=0.1
```

```
Accept-Charset: ISO-8859-1, utf-8;q=0.66, */*;q=0.66
```

```
Accept-Language: en
```

จากตัวอย่างนี้มีการ include ค่า option ใน header ทั้งหมด 4 ตัวคือ User-Agent, Accept, Accept-Charset, และ Accept-Language ซึ่งก็เป็นวิธีที่ดีวิธีหนึ่งที่มีการ Check ค่า Browser ว่ามาจากตัวเดียวกันหรือไม่ พิจารณา header ต่อไปนี้

```
GET / HTTP/1.1
```

```
Host: www.example.org
```

```
Cookie: PHPSESSID=12345
```

```
User-Agent: Mozilla/5.0 (compatible; IE 6.0 Microsoft Windows XP)
```

เราสามารถใส่ User-Agent ลงไปเพื่อรักษาความปลอดภัยได้โดยใช้ตัวอย่าง Code ดังต่อไปนี้

ตารางที่ 4.6 แสดงตัวอย่าง Code ที่มีการใช้งาน User-Agent ในการตรวจสอบ Session

```
<?php
session_start();

if (isset($_SESSION['HTTP_USER_AGENT']))
{
if ($_SESSION['HTTP_USER_AGENT'] != md5($_SERVER['HTTP_USER_AGENT']))
{
/* Prompt for password */
exit;
}
}
else
{
$_SESSION['HTTP_USER_AGENT'] = md5($_SERVER['HTTP_USER_AGENT']);
}
?>
```

จากนี้ไปผู้โจมตีต้องผ่าน 2 ขั้นตอนเสมอจึงจะทำ Session Hijacking ได้สำเร็จคือต้องมี SessionID เดียวกัน แล้วยังต้องมี User-Agent เหมือนกันด้วย ดังนั้นจึงเพิ่มความปลอดภัยได้ระดับหนึ่งทีเดียว แต่ถ้าจะให้ปลอดภัยกว่านี้เราก็อาจจะเพิ่ม Option อื่นลงไปด้วยก็ได้ ดังตัวอย่างต่อไปนี้

ตารางที่ 4.7 แสดงตัวอย่าง Code ที่มีการใช้งาน User-Agent และ Option อื่น ๆ ในการตรวจสอบ Session

```
<?php
session_start();

$fingerprint = 'SECRETSTUFF' . <br></br> $_SERVER['HTTP_USER_AGENT'] . <br></br>
$_SERVER['HTTP_ACCEPT_CHARSET'];
$_SESSION['fingerprint'] = md5($fingerprint <br></br> . session_id());
?>
```

จากตัวอย่างนี้ผู้โจมตีต้องมี 3 อย่างด้วยกันจึงสามารถทำ Session Hijacking ได้คือ ต้องมี SessionID เดียวกัน มี HTTP header เหมือนกัน และก็ต้องมี fingerprint ที่เหมือนกันด้วย

จากตัวอย่างที่ผ่านมา เราใช้ User-Agent หรือ Option อื่นๆใน HTTP Header มาช่วยในการตรวจสอบ แต่ก็อาจเป็นไปได้ที่ผู้โจมตีอาจสามารถเดาค่าต่างๆได้เช่นกัน สำหรับอีกวิธีที่น่าสนใจคือ การนำ token มาใช้ประกอบกับ URL ซึ่ง token นี้จะถูกใช้งานในทุกๆหน้า โดย token นี้ต้องไม่สามารถคาดเดาได้ง่าย โดยอาจนำมาจาก option ต่างๆใน HTTP Header เอง หรืออาจทำการสุ่มขึ้นมาก็ได้ แต่ก็แน่นอนอีกว่าผู้โจมตีก็ต้องพยายามดักจับค่าทั้งสองมาคือทั้ง PHPSESSID และ token ที่เราสร้างขึ้น ซึ่งก็ทำให้การโจมตียากขึ้นด้วยเช่นกัน

ตารางที่ 4.8 แสดงตัวอย่าง Code ที่มีการใช้งาน Token ในการตรวจสอบ Session

```
<?php
$token = md5(uniqid(rand(), TRUE));
$_SESSION['token'] = $token;
?>
```

4.4.5 ตัวอย่างการเข้าแย่งเพื่อครอบครองเซสชัน (Session Hijacking)

Ethic

“ การสาธิตการเจาะระบบและการ Injection ทั้งหมด ล้วนกระทำขึ้นเป็นกรณีศึกษา เพื่อให้เกิดความตระหนัก ถึงความสำคัญในการรักษาความปลอดภัยของระบบ Web Application และทราบถึงแนวคิดในการเจาะระบบของแฮกเกอร์ เพื่อให้เข้าใจ และสามารถนำไปใช้ปรับปรุงการป้องกันได้อย่างมีประสิทธิภาพเท่านั้น มิได้มีเจตนาเผยแพร่เพื่อให้ฝึกฝนหรือกระทำตามอันอาจก่อให้เกิดความเสียหายแก่ ทรัพย์สินของผู้อื่นๆได้ ในสาธิตตลอดจนการศึกษการเจาะระบบทั้งหมด กระทำการขึ้นเพื่อวัตถุประสงค์ทางการศึกษา และเมื่อเจาะระบบสำเร็จก็ถือว่าการศึกษาจบลง โดยไม่มีการแสวงหาผลประโยชน์เพิ่มเติมจากเว็บไซต์ที่สามารถเจาะได้”

รายละเอียดเกี่ยวกับเว็บไซต์

www.mobclub.net เป็นเว็บไซต์ที่ให้บริการสื่อมัลติมีเดียบนโทรศัพท์มือถือ เช่นบริการ download ข้อความ SMS MMS เป็นต้น

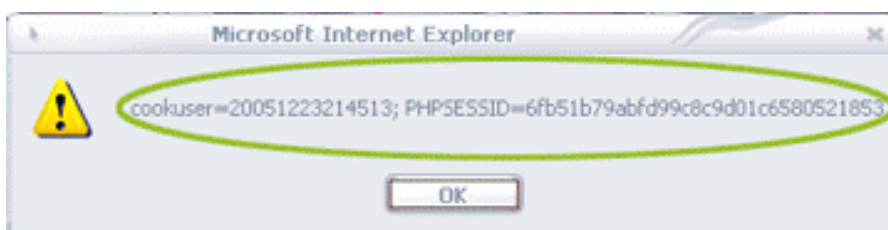
จุดอ่อนของเว็บไซต์

จากการตรวจสอบพบช่องโหว่ในส่วนของการแมปคอนเน็กชันระหว่างฝั่ง Client และ Server ทำให้สามารถทำ Session Hijacking ได้

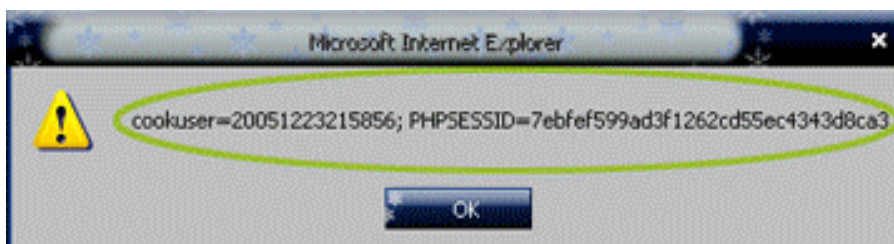
ขั้นตอนการเจาะระบบ

สำหรับตัวอย่างการเจาะระบบนี้ ได้ใช้คอมพิวเตอร์ 2 เครื่องในการทดสอบซึ่งสามารถสังเกตได้จากรูปภาพ โดยภาพบน เป็นคอมพิวเตอร์ในการทดสอบเครื่องที่ 1 ส่วนภาพล่าง เป็นคอมพิวเตอร์ในการทดสอบเครื่องที่ 2

1. เข้าสู่เว็บไซต์ mobiclub.net จากคอมพิวเตอร์ทั้งสองเครื่อง จากนั้น ดู cookie จะพบว่ามียู่ 2 ค่า คือ cookuser และ PHPSESSID ซึ่งจะเห็นได้ว่า ค่า cookuser เป็นตัวเลขที่ประกอบไปด้วย การรวมกันของ ปี-เดือน-วัน-ชั่วโมง-นาที-วินาที นั้นเอง



รูปที่ 4.17 ทำ Javascript Injection เพื่อสำรวจค่า cookuser และ PHPSESSID ของเครื่องที่ 1



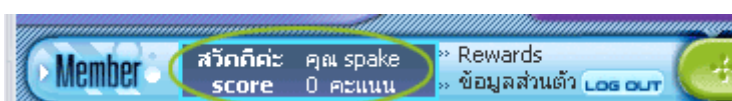
รูปที่ 4.18 ทำ Javascript Injection เพื่อสำรวจค่า cookuser และ PHPSESSID ของเครื่องที่ 2

2. ใช้คอมพิวเตอร์เครื่องแรก login เข้าสู่ระบบ



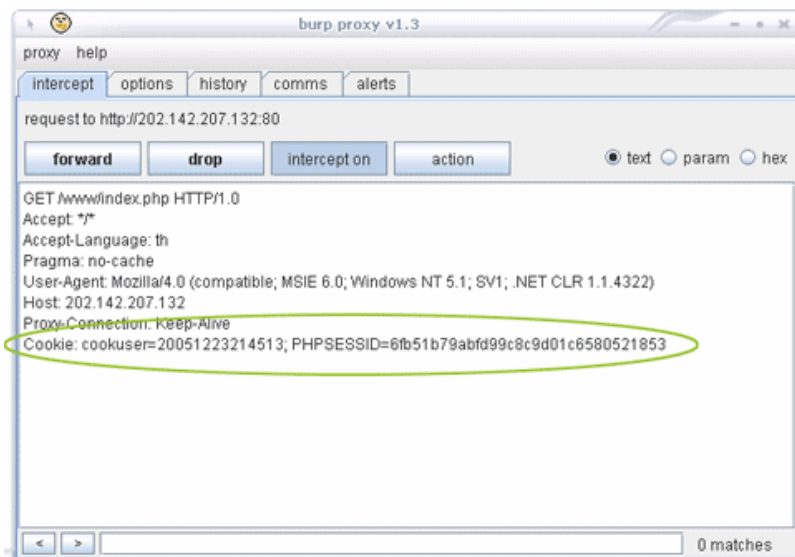
รูปที่ 4.19 Login เข้าสู่ระบบบนเครื่องที่ 1

3. เมื่อ login สำเร็จจะขึ้นกล่องแสดงคะแนนของผู้ใช้งานดังรูป

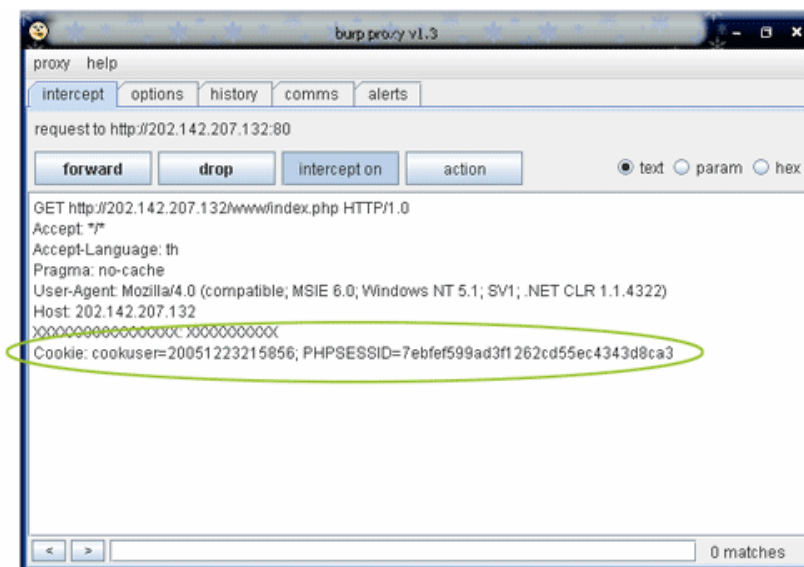


รูปที่ 4.20 Login สำเร็จบนเครื่องที่ 1

4. เปิดโปรแกรมที่สามารถดักจับ Header ของ HTTP ได้เช่น Burp Proxy จากนั้นให้ reload หน้าแรกของคอมพิวเตอร์ทั้ง 2 เครื่อง จากนั้นให้นำค่า cookuser และ PHPSESSID จากคอมพิวเตอร์เครื่องแรก ไปแทนที่ในเครื่องที่สอง

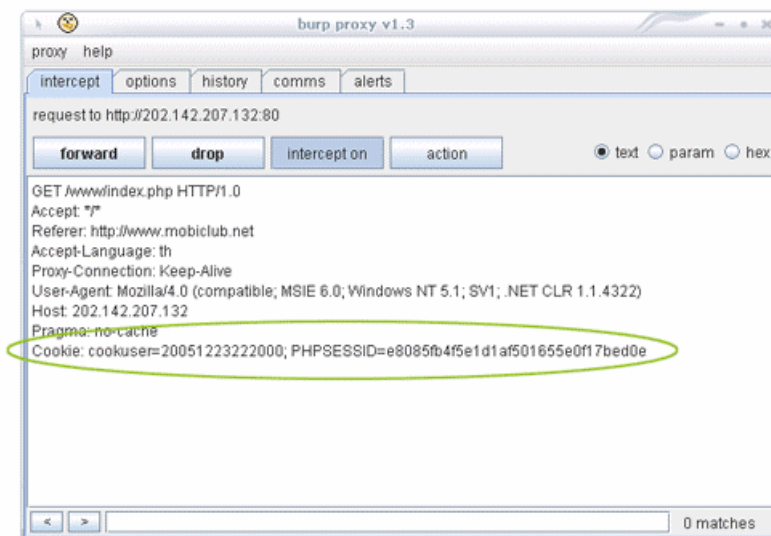


รูปที่ 4.21 ภาพแสดงการดักจับ Header ของเครื่องที่ 1

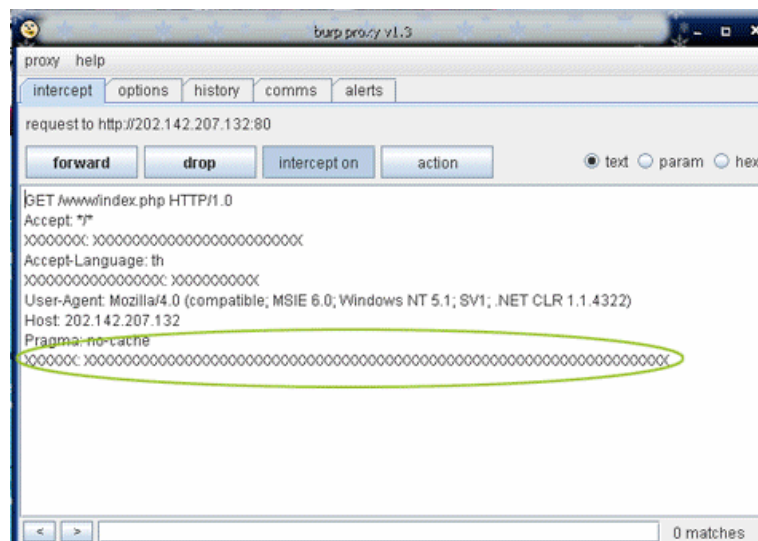


รูปที่ 4.22 ภาพแสดงการดักจับ Header ของเครื่องที่ 2

5. ทำตามข้อ 4 ทุกครั้งที่มีการส่ง Message จาก browser เพื่อร้องขอเว็บเพจจาก server

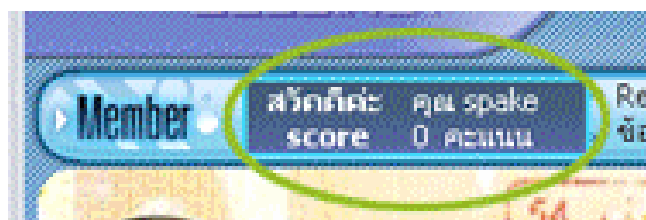


รูปที่ 4.23 ภาพแสดงการดักจับ Header ของเครื่องที่ 1



รูปที่ 4.24 ภาพแสดงการดักจับ Header ของเครื่องที่ 2

6. เมื่อไม่มี message ส่งออกไปร้องขอแล้วจะพบว่า คอมพิวเตอร์เครื่องที่ 2 สามารถได้ session เช่นเดียวกับ คอมพิวเตอร์เครื่องแรกเช่นกัน



รูปที่ 4.25 สถานะการเป็นสมาชิกบนเครื่องที่ 1



รูปที่ 4.26 สถานะการเป็นสมาชิกบนเครื่องที่ 2

4.5 ฟังก์ชันที่ควรใช้ในการเขียนโปรแกรมเพื่อเพิ่มความปลอดภัยต่อการถูกบุกรุก

สำหรับฟังก์ชันเหล่านี้ เป็นตัวอย่างฟังก์ชันที่มีประโยชน์ในการเขียนโปรแกรมเพื่อเพิ่มความปลอดภัยต่อการถูกบุกรุกเว็บไซต์ ซึ่งการใช้งานสามารถนำไฟล์นี้ไปเพิ่มในโปรแกรมได้เลย โดยใช้ฟังก์ชัน include ใน PHP ได้เลยดังนี้

```
Include("lancelot_code.php");
```

ฟังก์ชันในไฟล์ประกอบไปด้วย 9 ฟังก์ชันคือ

1. ฟังก์ชัน **CheckInput(\$sData, \$iMin, \$iMax)** - ฟังก์ชันนี้มีหน้าที่ตรวจสอบข้อมูลว่ามีขนาดตามที่กำหนดไว้หรือไม่ ถ้ามีขนาดตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีขนาดไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```
function CheckInput($sData, $iMin, $iMax) {
    $data = preg_match("/^(.){". $iMin. ", ". $iMax. "}$/", $sData);
    return $data;
}
```

2. ฟังก์ชัน **CheckEmpty(\$sName)** - ฟังก์ชันนี้มีหน้าที่ตรวจสอบว่ามีข้อมูลอยู่หรือไม่ ถ้ามีจะคืนค่าเป็น 1 ถ้าไม่มีจะคืนค่าเป็น 0

```
function CheckEmpty($sName)
{
    $name=split (" ", $sName);
    $count=count($name);
    $check=0;
    for($i=0;$i<$count;$i++)
```

```

    {
        if(!empty($name[$i]))
            $check++;
    }
    return $check;
}

```

3. ฟังก์ชัน **CheckEmail(\$sData)** - ฟังก์ชันนี้มีหน้าที่ตรวจสอบข้อมูลว่าเป็นรูปแบบในลักษณะ e-mail หรือไม่ ถ้ามีรูปแบบตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีรูปแบบไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```

function CheckEmail($sData) {
    $data = preg_match("/[a-z0-9|_|\.]+\@[a-z0-9|\.]+\.[a-z]{2,3}$/i", $sData);
    return $data;
}

```

4. ฟังก์ชัน **CheckUser(\$sData)** - ฟังก์ชันนี้มีหน้าที่ตรวจสอบข้อมูลว่ามีรูปแบบตามที่กำหนดไว้หรือไม่ โดยรูปแบบที่กำหนดไว้ต้องมีขนาดไม่เกิน 50 ตัวอักษร ถ้ามีรูปแบบตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีรูปแบบไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```

function CheckUser($sData) {
    $data = preg_match("/^[a-z0-9|_|-]{1,50}$/i", $sData);
    return $data;
}

```

5. ฟังก์ชัน **CheckPass(\$sData)** - ฟังก์ชันนี้มีหน้าที่ตรวจสอบรูปแบบของรหัสผ่านว่าถูกต้องตามรูปแบบหรือไม่ โดยรูปแบบที่ถูกต้องนั้นต้องมีความยาวตั้งแต่ 3 ถึง 12 ตัวอักษรเท่านั้น และจะเป็นตัวอะไรก็ได้ ถ้ามีรูปแบบตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีรูปแบบไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```

function CheckPass($sData) {
    $data = preg_match("/^[a-z0-9|_|-]{3,12}$/i", $sData);
}

```

```

return $data;
}

```

6. ฟังก์ชัน **CheckTelephone(\$sPhone)** – ฟังก์ชันนี้มีหน้าที่ตรวจสอบรูปแบบของเบอร์โทรศัพท์ว่าถูกต้องตามรูปแบบหรือไม่ โดยรูปแบบที่ถูกต้องนั้นต้องมีความยาว 9 ตัวอักษรเท่านั้น และต้องเป็นตัวเลขเท่านั้นด้วย ถ้ามีรูปแบบตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีรูปแบบไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```

function CheckTelephone($sPhone)
{
    $valid="[0-9]{9}";    $data = ereg($valid, $sPhone);
    return $data;
}

```

7. ฟังก์ชัน **Checkyear(\$years)** – ฟังก์ชันนี้มีหน้าที่ตรวจสอบรูปแบบของปีว่าถูกต้องตามรูปแบบหรือไม่ โดยรูปแบบที่ถูกต้องนั้นต้องมีความยาว 4 ตัวอักษรเท่านั้น และต้องเป็นตัวเลขเท่านั้นด้วย ถ้ามีรูปแบบตามที่กำหนดจะคืนค่าเป็น 1 แต่ถ้ามีรูปแบบไม่ตรงตามที่กำหนดจะคืนค่าเป็น 0

```

function Checkyear($years)
{
    $valid="[0-9]{4}";
    $data = ereg($valid, $years);
    return $data;
}

```

8. ฟังก์ชัน **hexentities(\$str)** – เป็นฟังก์ชันเพื่อช่วยแปลงค่าจากตัวอักษรธรรมดาให้เป็นค่า Hex Entity

```

function hexentities($str) {
    $return = "";
    for($i = 0; $i < strlen($str); $i++) {
        $return .= '&#x'.bin2hex(substr($str, $i, 1)).';';
    }
}

```

```

    }
    return $return;
}

```

9. ฟังก์ชัน **filter_tags(\$str)** - เป็นฟังก์ชันที่อนุญาตเฉพาะ Tag ที่กำหนดในการแสดงผล และการกรอง Script ออกจาก HTML Code โดย Tag ที่ต้องการอนุญาตถูกกำหนดไว้ที่ตัวแปร `allow_tags`

```

function filter_tags($str) {
    $allowed_tags = "<textarea>,<a>,<h1>,<h2>,<h3>,<h4>,<h5>,<h6>";
    $return = strip_tags($text,$allowed_tags);
    return $return;
}

```